

通俗易懂理解决策树算法、剪枝处理及 Python 代码实现

原创 ZEERO~ 于 2020-08-06 22:10:29 发布 阅读量 3k 收藏 31 点赞数 3

分类专栏: 机器学习 文章标签: 机器学习 决策树 剪枝 数据挖掘

版权

 机器学习 专栏收录该内容

0 订阅 11 篇文章 [订阅专栏](#)

通俗讲解决策树算法

- 一、算法概述
- 二、划分选择
 - 2.1 ID3 决策树算法与信息增益
 - 信息熵
 - 信息增益
 - 2.2 C4.5 算法与信息增益率
 - 增益率
 - 2.3 CART 决策树算法与基尼指数
- 三、剪枝处理
 - 预剪枝
 - 后剪枝
- 四、连续值处理
 - 二分法
- 五、python 代码实现
 - 5.1 创建数据集。
 - 5.2 计算数据集的信息熵（香农熵）
 - 5.2.1 辅助函数，统计样本中不同类别的数目
 - 5.2.2 计算信息熵
 - 5.3 根据属性对数据集进行分割
 - 5.3.1 辅助函数，找出数据集中比例占多数的类别
 - 5.3.2 根据离散属性进行分割
 - 5.3.3 根据连续属性进行分割
 - 5.4 选择最优属性
 - 5.5 创建决策树
 - 5.6 将决策树打印出来

 ZEERO~ [关注](#)

 3   31  0  分享 ... [专栏目录](#)

一、算法概述

决策树 (decision tree) 是一类常见的机器学习方法。一般地，一颗决策树包含一个根节点、若干个内部节点和若干个叶节点。**叶节点则对应决策结果**。决策树学习的目的是为了产生一颗泛化能力强，即处理未见示例能力强的决策树。

二、划分选择

一般而言，随着划分过程不断进行，我们希望决策树的分支节点所包含的样本尽可能属于同一类别，即节点的**纯度**(purity)越来越高。**根据划分方式的不同，也产生了不同的决策树算法。**

2.1 ID3 决策树算法与信息增益

信息增益是基于**信息熵**基础上的，因此先讲下信息熵是什么。

信息熵

“**信息熵**”是度量样本集合纯度最常用的一种指标。假定当前样本集合D中第k类样本所占比例为 p_k ($k=1,2,\dots,|y|$)，则D的信息熵定义为：

$$Ent(D) = - \sum_{k=1}^{|y|} p_k \log_2 p_k$$

假定我们只有2类，每类所占的比例都为1/2。则这个时候Ent(D)的值为1。假设这个时候有10类，每个类所占比例都是1/10，则Ent(D)的值为3.32。这里我们约定，当 $p=0$ 时， $\log_2 p=0$ 。我们这里也可看出来，熵值越大，则D的纯度越低。

信息增益

信息增益指的是用某个属性对样本集合进行划分之后样本信息熵**减小**的值。

假定离散属性a有V个可能的取值 $\{a^1, a^2, \dots, a^V\}$ ，若使用a来对样本集D进行划分，则会产生V个分支节点。其中第v个分支节点包含了D中所有在属性a上取值为 a^v 的样本，记为 D^v 。我们可由公式计算出每个分支节点的信息熵值。再考虑到不同的分支节点所包含的样本数不同，我们给分支节点赋予权重 $|D^v| / |D|$ ，即样本数越多的分支节点的影响越大，于是可计算出用属性a对样本集D进行划分所获得的“信息增益”计算公式如下：

$$Gain(D, a) = Ent(D) - \sum_{v=1}^V \frac{|D^v|}{|D|} Ent(D^v)$$

一般而言，**信息增益越大**，则意味着使用属性a来进行划分所获得的“纯度”提升越大。因此，我们可用信息增益来进行决策树的划分属性选择。著名的ID3决策树学习算法就是以信息增益为准则来选择属性划分。



ZEERO~

关注

👍 3



★ 31

💬 0

🔗 分享



专栏目

2.2 C4.5 算法与信息增益率

上节所讲信息增益算法其实有个很大的缺点。缺点在于，信息增益准则对可取值数目较多的属性有所偏好。这也很容易理解，如果属性可取值数目较多，用这个属性划分数据集后的分支节点的纯度相对来说会更大。取极限情况来说，假设对每个样本都进行编号，则使用编号这个属性进行划分，只需一次便可产生一颗完整的决策树。为减少这种偏好可能带来的不利影响，著名的C4.5决策树算法不直接使用信息增益，而是使用增益率来选择最优划分属性。

增益率

增益率的定义为：

$$Gain_ratio(D,a) = \frac{Gain(D,a)}{IV(a)}$$

其中，

$$IV(a) = - \sum_{v=1}^V \frac{|D^v|}{|D|} \log_2 \frac{|D^v|}{|D|}$$

IV (a) 称为属性a的固有值。属性a的可能取值数目越多（即V越大），则 IV (a) 的值通常也会越大。需要注意的是，C4.5算法对可取值数目较少的属性会有所偏好。因此，C4.5算法并不是直接选择增益率最大的候选划分属性，而是使用了一个启发式的算法：先从候选划分属性中找出信息增益高于平均水平的属性，再从中选择增益率最高的。

2.3 CART 决策树算法与基尼指数

（为了克服 ID3 算法和 C4.5 算法的缺点，不知道是不是，笔者瞎编的），CART（Classification and RegressionTree）则是使用基尼指数来选择划分属性。我们这里用基尼值来度量数据D的纯度：

$$Gini(D) = \sum_{k=1}^{|y|} \sum_{k' \neq k} p_k p_{k'} = 1 - \sum_{k=1}^{|y|} p_k^2$$

直观来说，Gini(D)反应了从数据集D中随机抽取两个样本，其类别标记不一致的概率。因此，Gini (D) 越小，则数据集D的纯度越高。属性a的基尼指数定义为：

$$Gini_{index}(D,a) = \sum_{v=1}^V \frac{|D^v|}{|D|} Gini(D^v)$$

于是，我们在候选集属性集合A中，选择那个使得划分后基

 ZEERO~ 关注

 3   31  0  分享 ... 专栏目

三、剪枝处理

剪枝 (pruning) 是决策树学习算法对付“过拟合”的主要手段。在决策树学习过程中，为了尽可能正确分类训练样本，节点划分过程将不断重复，有时会造成决策树分支过多，这时就可能因训练样本学的“太好”了，以致于把训练集自身的一些特点当做所有数据都具有的一般性质而导致过拟合。因此，可通过主动去掉一些分支来降低过拟合风险。决策树剪枝的基本策略有“预剪枝”和“后剪枝”。

预剪枝

预剪枝是指在决策树生成过程中，对每个结点在划分前先进行估计，若当前节点的划分不能带来决策树泛化 **性能** 的提升，则停止划分当前节点并将当前节点标记为叶节点。现在问题来了？**如何判断决策树泛化性能是否提升了呢？**
答案在验证集中进行测试。学过机器学习的同学都知道，数据集可分为训练集和测试集。我们在训练集中可预留出一部分数据作为验证集。通过使用决策树对验证集进行分类，若验证集分类准确率上升，我们则可认为决策树泛化性能提升了。

后剪枝

后剪枝则是先从训练集生成一颗完整的决策树，然后自底向上地对非叶节点进行考察。若将该节点对应的子树替换为叶节点能带来决策树泛化性能提升，则将该子树替换为叶结点。一般情形下，**后剪枝决策树的欠拟合风险很小，泛化性能往往优于预剪枝决策树。**但是后剪枝过程是在生成决策树之后进行的，并且要自底向上地对树中所有非叶节点进行逐一考察，因此其训练时间开销比未剪枝决策树和预剪枝决策树都要大很多。**若不考虑计算开销影响，一般往往选择后剪枝方式进行处理。**

四、连续值处理

在现实学习任务中会遇到连续属性的情况，这种情况下需如何处理呢？最简单的策略是采用二分法对连续属性进行处理。这也是 C4.5 决策树算法中采用的机制。

二分法

给定样本集 D 和连续属性 a，假定 a 在 D 上出现了 n 个不同的取值，将这些值从小到大进行排序，记为 $\{a^1, a^2, \dots, a^n\}$ 。基于划分点 t 可将 D 分为子集 D_t^- 和 D_t^+ 。其中 D_t^- 包含那些在属性 a 上取值不大于 t 的样本，而 D_t^+ 则包含那些在属性 a 上取值大于 t 的样本。显然，对相邻的属性取值 a_i 与 a_{i+1} 来说，t 在区间 $[a_i, a_{i+1})$ 中取任意值所产生的划分结果相同。因此，对连续属性 a，我们可以考察包含 n-1 个元素的候选划分点集合：

$$T_a = \left\{ \frac{a^i + a^{i+1}}{2} \mid 1 \leq i \leq n - 1 \right\}$$

即把区间的中位点作为候选划分点，然后，我们就可象离散属性值一样来考察这些划分点，选取最优的划分点进行样本集合的划分。我们可对公式稍加改造：

$$Gain(D, a, t) = \max_{t \in T_a} Gain(D, a, t) = \max_{t \in T_a} Ent(D) - \sum_{\gamma \in \{-, +\}} \frac{|D_t^\gamma|}{|D|} Ent(D_t^\gamma)$$

其中 Gain(D,a,t)是样本集 D 基于划分点 t 二分后的信息增益。
需要注意的是，若当前节点划分属性为连续属性，该属性还可

 ZEERO~

关注

👍 3

👎

🌟 31

💬 0

🔗 分享

⋮

专栏目

五、python代码实现

决策树在sklearn中有库 **函数** 可以直接调用实现。我们这里讲下使用python从底层进行实现。这里我们实现的是C4.5决策树算法。ID3算法中没有对连续属性值的处理。将增益率换为信息增益，则退化为了ID3决策树算法，替换为基尼指数则进化为了CART决策树算法。可以自己实现。

声明，代码搬运至某个github项目，然而找不到地址了，如果作者看到这篇文章，请联系我加上去，这里先用下，谢谢。笔者跑通后加了许多注释和讲解。

5.1 创建数据集。

数据集直接在python中创建，省去许多不必要的麻烦。

```
1 import operator
2 from math import log, sqrt
3
4 def createDataSet(): # 生成8个样本，此处的labels不是标签，是属性。dataSet的最后一列才是对应的标签是男或者女。数据集是一个多维列表
5     dataSet = [[1, '长', '粗', '男'],
6                [2, '短', '粗', '男'],
7                [3, '短', '粗', '男'],
8                [4, '长', '细', '女'],
9                [5, '短', '细', '女'],
10               [6, '短', '粗', '女'],
11               [7, '长', '粗', '女'],
12               [8, '长', '粗', '女']]
13     labels = ['序号', '头发', '声音'] # three features
14     return dataSet, labels
```

我们可以看出这里生成了8个样本，共有3个属性，分别是“序号”，“头发”，“声音”。

5.2 计算数据集的信息熵（香农熵）

5.2.1 辅助函数，统计样本中不同类别的数目

```
1 def classCount(dataSet): #最后得到一个字典{'男': 3, '女': 5}
2     labelCount = {}
3     for one in dataSet:
4         if one[-1] not in labelCount.keys():
5             labelCount[one[-1]] = 0
6             labelCount[one[-1]] += 1
7     return labelCount
```

5.2.2 计算信息熵



ZEERO~

关注

👍 3



★ 31

💬 0

🔗 分享



专栏目

```

1 def calcShannonEntropy(dataSet):#直接计算数据集的信息熵
2     labelCount = classCount(dataSet)
3     numEntries = len(dataSet)#数据总长度: 8
4     Entropy = 0.0
5     for i in labelCount:
6         prob = float(labelCount[i]) / numEntries
7         Entropy -= prob * log(prob, 2)
8     return Entropy

```

5.3 根据属性对数据集进行分割

5.3.1 辅助函数，找出数据集中比例占多数的类别

```

1 def majorityClass(dataSet):#找出数据集中比例占多数的类别
2     labelCount = classCount(dataSet)
3     sortedLabelCount = sorted(labelCount.items(), key=operator.itemgetter(1), reverse=True)
4     return sortedLabelCount[0][0]

```

5.3.2 根据离散属性进行分割

```

1 def splitDataSet(dataSet, i, value):#取出并返回数据集第i个维度上值为value的子集，i为属性。并且返回的数据去除了第i维
2     subDataSet = []
3     for one in dataSet:
4         if one[i] == value:
5             reduceData = one[:i]
6             reduceData.extend(one[i + 1:])#这两行操作的作用是去除第i维数据
7             subDataSet.append(reduceData)
8     return subDataSet

```

5.3.3 根据连续属性进行分割

```

def splitContinuousDataSet(dataSet, i, value,
                           direction): # split the data according the value, i was axis ,direction 0 is >= and direction 1 is <=
1     subDataSet = []
2     for one in dataSet:
3         if direction == 0:
4             if one[i] > value:#direction为0，是取大于的值，direction为1，则是取负方向的值
5                 reduceData = one[:i]
6                 reduceData.extend(one[i + 1:])
7                 subDataSet.append(reduceData)
8         if direction == 1:
9             if one[i] <= value:
10

```



ZEERO~

关注

👍 3



🌟 31

💬 0

🔗 分享



专栏目

```

11
12         reduceData = one[:i]
13         reduceData.extend(one[i + 1:])
14         subDataSet.append(reduceData)
15     return subDataSet

```

5.4 选择最优属性

对所有属性进行分割之后，我们选择最优属性。函数最后的返回值为取到的最优属性和此时的最优属性分割值。

```

1  #注意，这里是直接选择最优增益率的属性，没有使用上文中所说的启发式搜索，即先从划分属性中找出信息增益高于平均水平的属性，再从中选择增益率最高的。
2  def chooseBestFeat(dataSet, labels):
3      baseEntropy = calcShannonEntropy(dataSet) #先计算数据集总体的信息熵
4      bestFeat = 0 #先假定bestFeat为第0维的属性
5      baseGainRatio = -1
6      numFeats = len(dataSet[0]) - 1 # the number of features 特征的个数
7      bestSplitDic = {}
8      i = 0
9      print('dataSet[0]:' + str(dataSet[0]))
10     for i in range(numFeats):
11         featVals = [example[i] for example in dataSet] # 将dataset中的第i列数据取出
12         # print('chooseBestFeat:' + str(i))
13         if type(featVals[0]).__name__ == 'float' or type(featVals[0]).__name__ == 'int':# 判断属性是否为连续属性。对连续值进行划分选择，else则是对离散值进行划分选择
14             j = 0
15             sortedFeatVals = sorted(featVals)#对选中的这列属性值进行升序排列
16             splitList = [] #储存二分法的连续属性的节点(这里为7个值)
17             for j in range(len(featVals) - 1):
18                 splitList.append((sortedFeatVals[j] + sortedFeatVals[j + 1]) / 2.0)
19             for j in range(len(splitList)):
20                 newEntropy = 0.0
21                 gainRatio = 0.0
22                 splitInfo = 0.0
23                 value = splitList[j] #对二分节点依次进行遍历
24                 subDataSet0 = splitContinuousDataSet(dataSet, i, value, 0)#大于value的数据集
25                 subDataSet1 = splitContinuousDataSet(dataSet, i, value, 1)#小于value的数据集
26                 prob0 = float(len(subDataSet0)) / len(dataSet)
27                 newEntropy -= prob0 * calcShannonEntropy(subDataSet0)
28                 prob1 = float(len(subDataSet1)) / len(dataSet)
29                 newEntropy -= prob1 * calcShannonEntropy(subDataSet1)
30                 splitInfo -= prob0 * log(prob0, 2)
31                 splitInfo -= prob1 * log(prob1, 2)
32                 gainRatio = float(baseEntropy - newEntropy) / splitInfo
33

```



ZEERO~

关注

👍 3



🌟 31

💬 0

🔗 分享



专栏目

```

34         print('Iva ' + str(j) + ':' + str(splitInfo))
35         if gainRatio > baseGainRatio:
36             baseGainRatio = gainRatio
37             bestSplit = j
38             bestFeat = i
39         bestSplitDic[labels[i]] = splitList[bestSplit]
40     else: #对离散属性值进行划分
41         uniqueFeatVals = set(feetVals)
42         GainRatio = 0.0
43         splitInfo = 0.0
44         newEntropy = 0.0
45         for value in uniqueFeatVals:
46             subDataSet = splitDataSet(dataSet, i, value)
47             prob = float(len(subDataSet)) / len(dataSet)
48             splitInfo -= prob * log(prob, 2)
49             newEntropy -= prob * calcShannonEntropy(subDataSet)
50         gainRatio = float(baseEntropy - newEntropy) / splitInfo
51         if gainRatio > baseGainRatio:
52             bestFeat = i
53             baseGainRatio = gainRatio
54     if type(dataSet[0][bestFeat]).__name__ == 'float' or type(dataSet[0][bestFeat]).__name__ == 'int':
55         bestFeatValue = bestSplitDic[labels[bestFeat]]
56         ##bestFeatValue=labels[bestFeat]+'<='+str(bestSplitValue)
57     if type(dataSet[0][bestFeat]).__name__ == 'str':
58         bestFeatValue = labels[bestFeat]
59     return bestFeat, bestFeatValue

```

5.5 创建决策树

```

def createTree(dataSet, labels):
    classList = [example[-1] for example in dataSet] #直接取了数据集中的最后一列作为标签
    1   if len(set(classList)) == 1: #这四行为递归终止条件。若类别中只剩下一项时，停止递归；
    2       return classList[0][0]
    3   if len(dataSet[0]) == 1: #若数据集中只剩下一项属性值时，直接根据按比例生成树
    4       return majorityClass(dataSet)
    5   Entropy = calcShannonEntropy(dataSet) #计算数据集的信息熵
    6   bestFeat, bestFeatLabel = chooseBestFeat(dataSet, labels) #选择当前数据集中的最优属性
    7   print('bestFeat:' + str(bestFeat) + '--' + str(labels[bestFeat]) + ', bestFeatLabel:' + str(bestFeatLabel))
    8   myTree = {labels[bestFeat]: {}} #建立一个集合用来存放树结构
    9   subLabels = labels[:bestFeat]
    10  subLabels.extend(labels[bestFeat + 1:]) #这两行的作用是将属性Labels中除最优属性外的其他属性拿出来
    11  print('subLabels:' + str(subLabels))
    12  if type(dataSet[0][bestFeat]).__name__ ==
    13

```



ZEERO~

关注



3



31



0



分享



专栏目


```

14
15
16     featVals = [example[bestFeat] for example in dataSet]
17     uniqueVals = set(featVals)
18     print('uniqueVals:' + str(uniqueVals))
19     for value in uniqueVals:#递归调用
20         reduceDataSet = splitDataSet(dataSet, bestFeat, value)
21         print('reduceDataSet:' + str(reduceDataSet))
22         myTree[labels[bestFeat]][value] = createTree(reduceDataSet, subLabels)
23 if type(dataSet[0][bestFeat]).__name__ == 'int' or type(dataSet[0][bestFeat]).__name__ == 'float':
24     value = bestFeatLabel
25     #将数据集根据最优属性值进行划分，划分成两个子集
26     greaterDataSet = splitContinuousDataSet(dataSet, bestFeat, value, 0)
27     smallerDataSet = splitContinuousDataSet(dataSet, bestFeat, value, 1)
28     print('greaterDataSet:' + str(greaterDataSet))
29     print('smallerDataSet:' + str(smallerDataSet))
30     print('== ' * len(dataSet[0]))
31     myTree[labels[bestFeat]]['>' + str(value)] = createTree(greaterDataSet, subLabels)
32     print(myTree)
33     print('== ' * len(dataSet[0]))
34     myTree[labels[bestFeat]]['<=' + str(value)] = createTree(smallerDataSet, subLabels)
return myTree

```

5.6 将决策树打印出来

```

1 if __name__ == '__main__':
2     dataSet, labels = createDataSet()
3     tree=createTree(dataSet, labels)
4     print(tree)

```

最后打印的结果如下所示：

```

1 {'序号': {'>7.5': '女', '<=7.5': {'头发': {'短': {'声音': {'细': '女', '粗': '男'}}}, '长': {'声音': {'细': '女', '粗': '男'}}}}}

```

全文到此结束。如果觉得对您有用，请帮忙点个赞，谢谢。如果有不对之处，还请指出，谢谢。

声明，决策树理论讲解部分主要参考自周志华的《机器学习》一书。另外，参考资料也——列举如下，如有异议，请直接联系笔者修改或删除。

参考资料：

[1]:《机器学习》周志华

决策树的剪枝



ZEERO~

关注

👍 3



★ 31

💬 0

🔗 分享



📖 专栏目